SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: MLA0304
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. Dumpala Prasanth

QUESTIONS: 10 **Total Marks: 50**

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Solution:
Procedure:
1. Download and install Anaconda Navigator from the official Anaconda website and complete the setup wizard.
2. Open Anaconda Prompt and create a new virtual environment: conda create -n rl_env python=3.10.
3. Activate the environment: conda activate rl_env.
4. Install the required libraries using pip/conda: TensorFlow, Keras, Gymnasium, NumPy, Matplotlib.
5. Launch Jupyter Notebook from Anaconda Navigator using the rl_env environment.
6. Verify the installation by importing each library and printing its version.

7. Run a simple Gymnasium environment to confirm the RL setup works end-to-end.

Program:

```
# Step 1: Create and activate environment (run in Anaconda Prompt)
conda create -n rl_env python=3.10 -y
conda activate rl_env

# Step 2: Install required packages
pip install tensorflow keras gymnasium numpy matplotlib

# Step 3: Verification script (run inside Jupyter Notebook / Python)
import tensorflow as tf
import keras
import gymnasium as gym
import numpy as np
import matplotlib

print("TensorFlow version :", tf.__version__)
print("Keras version       :", keras.__version__)
print("Gymnasium version   :", gym.__version__)
print("NumPy version       :", np.__version__)
print("Matplotlib version  :", matplotlib.__version__)

# Step 4: Quick sanity check using a simple RL environment
env = gym.make("CartPole-v1")
state, info = env.reset()
print("Initial state:", state)

action = env.action_space.sample()
next_state, reward, terminated, truncated, info = env.step(action)
print("Action taken:", action)
print("Next state:", next_state, "Reward:", reward)
env.close()
```

Output / Result:

All libraries import without errors and print their version numbers. The CartPole environment resets successfully, returns an initial state vector of 4 values, and a sample action produces a next state and a reward of 1.0, confirming that the Reinforcement Learning environment is correctly installed and configured.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Solution:

Procedure:

1. Import the gymnasium library.
2. Create each environment using gym.make() with the appropriate environment id.
3. Reset the environment to obtain the initial state/observation.
4. Sample random actions from the action space and step through the environment.
5. Observe the returned state, reward, terminated, truncated, and info values for each step.
6. Repeat the loop for a fixed number of steps or until the episode ends, then close the environment.

Program:

```
import gymnasium as gym

def run_environment(env_name, steps=10):
    print(f"\n--- Running {env_name} ---")
    env = gym.make(env_name)
    state, info = env.reset()
    print("Initial state:", state)

    total_reward = 0
    for step in range(steps):
        action = env.action_space.sample()          # random action
        next_state, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        print(f"Step {step+1}: action={action}, state={next_state}, "
              f"reward={reward}, done={terminated or truncated}")
        if terminated or truncated:
            state, info = env.reset()
        else:
            state = next_state

    print(f"Total reward collected in {env_name}: {total_reward}")
    env.close()

run_environment("FrozenLake-v1")
run_environment("CartPole-v1")
run_environment("MountainCar-v0")
```

Output / Result:

Each environment (FrozenLake, CartPole, MountainCar) initializes correctly and returns its state space on reset. Random actions produce a sequence of states, rewards, and done flags for every step, and the program prints the total reward accumulated per environment, demonstrating the state-action-reward-episode cycle of Reinforcement Learning.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Solution:

Procedure:

1. Define an Environment class that maintains the current state and defines a step() function returning next state, reward, and done flag.
2. Define an Agent class that holds a simple policy (e.g., random or rule-based) to choose an action given a state.
3. Run an interaction loop where the agent observes the state, selects an action using its policy, and the environment returns the next state and reward.
4. Accumulate the total reward over the episode and print the trajectory.
5. Repeat for multiple episodes to observe the agent-environment interaction.

Program:

```
import random

class Environment:
    def __init__(self):
```

```
        self.state = 0
        self.goal = 5

    def reset(self):
        self.state = 0
        return self.state

    def step(self, action):
        # action: 0 = move left, 1 = move right
        self.state += 1 if action == 1 else -1
        self.state = max(0, min(self.state, self.goal))
        done = self.state == self.goal
        reward = 10 if done else -1
        return self.state, reward, done


class Agent:
    def __init__(self, action_space):
        self.action_space = action_space
        self.policy = "random"

    def choose_action(self, state):
        return random.choice(self.action_space)


env = Environment()
agent = Agent(action_space=[0, 1])

for episode in range(3):
    state = env.reset()
    total_reward = 0
    trajectory = [state]
    done = False
    while not done:
        action = agent.choose_action(state)
        state, reward, done = env.step(action)
        total_reward += reward
        trajectory.append(state)

    print(f"Episode {episode+1}: trajectory={trajectory}, total_reward={total_reward}")
```

Output / Result:
The agent repeatedly interacts with the environment, choosing actions based on its policy and receiving states and rewards in return. Each episode prints the sequence of states visited (trajectory) and the total reward earned, illustrating the core state-action-reward-policy loop of the Reinforcement Learning framework.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Solution:
Procedure:
1. Define the set of states, actions, transition probabilities, and reward function of the MDP.
2. Represent the transition model as a dictionary mapping (state, action) to a list of (probability, next_state, reward) tuples.
3. Define a policy that maps each state to an action.

4. Simulate the MDP by sampling next states according to the transition probabilities for a number of time steps.
5. Record and print the sequence of states, actions, and rewards obtained during the simulation.

Program:

```
import random

# States and actions
states = ["S0", "S1", "S2", "Goal"]
actions = ["A0", "A1"]

# Transition model: (state, action) -> list of (probability, next_state, reward)
transitions = {
    ("S0", "A0"): [(0.8, "S1", -1), (0.2, "S0", -1)],
    ("S0", "A1"): [(0.5, "S2", -1), (0.5, "S0", -1)],
    ("S1", "A0"): [(1.0, "Goal", 10)],
    ("S1", "A1"): [(0.7, "S2", -1), (0.3, "S1", -1)],
    ("S2", "A0"): [(0.6, "Goal", 10), (0.4, "S2", -1)],
    ("S2", "A1"): [(1.0, "S1", -1)],
    ("Goal", "A0"): [(1.0, "Goal", 0)],
    ("Goal", "A1"): [(1.0, "Goal", 0)],
}

# Simple fixed policy
policy = {"S0": "A0", "S1": "A0", "S2": "A0", "Goal": "A0"}

def simulate_mdp(start_state="S0", max_steps=10):
    state = start_state
    total_reward = 0
    for t in range(max_steps):
        action = policy[state]
        outcomes = transitions[(state, action)]
        probs = [o[0] for o in outcomes]
        next_state, reward = random.choices(
            [(o[1], o[2]) for o in outcomes], weights=probs, k=1
        )[0]
        print(f"t={t}: state={state}, action={action} -> next_state={next_state},
reward={reward}")
        total_reward += reward
        state = next_state
        if state == "Goal":
            break
    print("Total reward:", total_reward)

simulate_mdp()
```

Output / Result:
The simulation prints the sequence of (state, action, next_state, reward) transitions generated according to the transition probabilities, terminating once the Goal state is reached, and reports the cumulative reward obtained under the fixed policy - demonstrating sequential decision-making in a Markov Decision Process.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Solution:

Procedure:
1. Define a small MDP with a set of states, actions, transition probabilities, and rewards.
2. Initialize the state-value function V(s) to zero for all states.
3. Implement the Bellman Expectation Equation to iteratively update V(s) under a given policy.
4. Implement the Bellman Optimality Equation to compute V*(s) by taking the maximum over actions.
5. Iterate the update rule until the value function converges (change below a small threshold).
6. Print the value function after each iteration and observe convergence.

Program:

```
import numpy as np

states = ["S0", "S1", "S2"]
actions = ["A0", "A1"]
gamma = 0.9

# transitions[state][action] = list of (prob, next_state, reward)
transitions = {
    "S0": {"A0": [(1.0, "S1", 5)],  "A1": [(1.0, "S2", 2)]},
    "S1": {"A0": [(1.0, "S2", 3)],  "A1": [(1.0, "S0", 0)]},
    "S2": {"A0": [(1.0, "S2", 0)],  "A1": [(1.0, "S1", 1)]},
}

V = {s: 0.0 for s in states}

def bellman_optimality_update(V):
    new_V = {}
    for s in states:
        action_values = []
        for a in actions:
            q = sum(p * (r + gamma * V[s_next]) for p, s_next, r in transitions[s][a])
            action_values.append(q)
        new_V[s] = max(action_values)
    return new_V

print("Iter 0:", V)
for i in range(1, 21):
    new_V = bellman_optimality_update(V)
    delta = max(abs(new_V[s] - V[s]) for s in states)
    V = new_V
    print(f"Iter {i}: {V}  (delta={delta:.5f})")
    if delta < 1e-4:
        print("Converged at iteration", i)
        break

print("\nOptimal state values:", V)
```

Output / Result:
The state-value estimates for S0, S1, and S2 are printed after every iteration and gradually stabilize as the value iteration progresses. The maximum change (delta) between successive iterations shrinks toward zero, and the loop terminates once delta falls below the convergence threshold, giving the optimal state-value function V*(s) computed via the Bellman Optimality Equation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Solution:

Procedure:

1. Define a simple environment with several actions, each having a fixed but unknown mean reward.
2. Implement a Random strategy that always picks an action uniformly at random.
3. Implement a Greedy strategy that always picks the action with the highest estimated value so far.
4. Implement an epsilon-Greedy strategy that picks a random action with probability epsilon and the best-known action otherwise.
5. Run each strategy for a fixed number of steps, updating the estimated action values after every reward.
6. Compare the cumulative reward obtained by each strategy.

Program:

```
import numpy as np

np.random.seed(0)
n_actions = 5
true_means = np.random.normal(0, 1, n_actions)

def get_reward(action):
    return np.random.normal(true_means[action], 1)

def run_strategy(strategy, steps=1000, epsilon=0.1):
    Q = np.zeros(n_actions)
    N = np.zeros(n_actions)
    total_reward = 0
    for t in range(steps):
        if strategy == "random":
            action = np.random.randint(n_actions)
        elif strategy == "greedy":
            action = np.argmax(Q)
        elif strategy == "epsilon_greedy":
            action = np.random.randint(n_actions) if np.random.rand() < epsilon else
np.argmax(Q)

        reward = get_reward(action)
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
        total_reward += reward
    return total_reward

for strategy in ["random", "greedy", "epsilon_greedy"]:
    total = run_strategy(strategy)
    print(f"{strategy:15s} -> total reward over 1000 steps: {total:.2f}")
```

Output / Result:

The program prints the total reward accumulated by the Random, Greedy, and epsilon-Greedy strategies over 1000 steps. Typically, epsilon-Greedy achieves the highest cumulative reward because it balances exploring uncertain actions with exploiting the best-known action, Greedy performs inconsistently because it can get stuck on a suboptimal action early on, and Random performs the worst since it never exploits learned information.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Solution:
Procedure:
1. Define a k-armed bandit where each arm returns a reward sampled from a normal distribution with a fixed unknown mean.
2. Implement the epsilon-Greedy algorithm that explores randomly with probability epsilon and exploits the best arm otherwise.
3. Implement the UCB algorithm that selects the arm maximizing Q(a) plus an exploration bonus based on how rarely it has been tried.
4. Run both algorithms for a fixed number of time steps, updating estimated action values after each pull.
5. Record and plot/print the cumulative reward over time for both algorithms and compare their performance.

Program:

```
import numpy as np

np.random.seed(1)
k = 10
true_means = np.random.normal(0, 1, k)
steps = 1000

def pull(arm):
    return np.random.normal(true_means[arm], 1)

def epsilon_greedy(epsilon=0.1):
    Q = np.zeros(k); N = np.zeros(k)
    rewards = []
    for t in range(steps):
        if np.random.rand() < epsilon:
            a = np.random.randint(k)
        else:
            a = np.argmax(Q)
        r = pull(a)
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards.append(r)
    return np.cumsum(rewards)

def ucb(c=2):
    Q = np.zeros(k); N = np.zeros(k)
    rewards = []
    for t in range(1, steps + 1):
        if 0 in N:
            a = np.argmin(N)          # try each arm once first
        else:
            ucb_values = Q + c * np.sqrt(np.log(t) / N)
            a = np.argmax(ucb_values)
        r = pull(a)
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards.append(r)
    return np.cumsum(rewards)
```

```
eg_cum = epsilon_greedy()
ucb_cum = ucb()

print("Cumulative reward (epsilon-Greedy) after 1000 steps:", round(eg_cum[-1], 2))
print("Cumulative reward (UCB) after 1000 steps:           ", round(ucb_cum[-1], 2))
```

Output / Result:
Both algorithms print their cumulative reward after 1000 pulls of the 10-armed bandit. UCB generally achieves a higher or comparable cumulative reward to epsilon-Greedy because its confidence-bound term drives more systematic exploration of less-tried arms early on, converging faster to the arm with the highest true mean reward.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Solution:
Procedure:
1. Create a simple environment (e.g., Gymnasium's CartPole) and an agent that selects actions randomly or using a basic epsilon-Greedy rule.
2. Run the agent for a number of episodes, recording the total reward obtained in each episode.
3. Compute the cumulative reward across episodes.
4. Use Matplotlib to plot the per-episode reward and the cumulative reward over episodes.
5. Analyze the plotted graphs to comment on the agent's learning performance.

Program:
```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

env = gym.make("CartPole-v1")
n_episodes = 100
episode_rewards = []

for ep in range(n_episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    while not done:
        action = env.action_space.sample()
        state, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        done = terminated or truncated
    episode_rewards.append(total_reward)

env.close()

cumulative_rewards = np.cumsum(episode_rewards)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(episode_rewards)
plt.xlabel("Episode")
```

```
plt.ylabel("Reward")
plt.title("Episode Reward")

plt.subplot(1, 2, 2)
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward over Episodes")

plt.tight_layout()
plt.savefig("reward_plots.png")
plt.show()
```

Output / Result:
Two graphs are generated: the first shows the reward obtained in every episode (fluctuating since actions are random), and the second shows the cumulative reward steadily increasing across episodes. The plots are saved as reward_plots.png and displayed, giving a clear visual summary of the agent's learning performance over time.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Solution:
Procedure:
1. Import TensorFlow and Keras and define the input dimension (state size) and output dimension (number of actions).
2. Build a Sequential feed-forward neural network with one or two hidden Dense layers using ReLU activation.
3. Add an output layer with linear activation representing the estimated Q-value / value for each action.
4. Compile the model with an optimizer (e.g., Adam) and a loss function (e.g., Mean Squared Error).
5. Generate sample state-target pairs and train the network for a few epochs to demonstrate value-function approximation.
6. Use the trained model to predict Q-values for a new state.

Program:
```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

state_size = 4     # e.g., CartPole observation size
n_actions = 2       # e.g., CartPole action size

model = keras.Sequential([
    layers.Input(shape=(state_size,)),
    layers.Dense(24, activation="relu"),
    layers.Dense(24, activation="relu"),
    layers.Dense(n_actions, activation="linear"),
])

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss="mse")
```

```
model.summary()

# Dummy training data: random states and target Q-values
X_train = np.random.rand(200, state_size)
y_train = np.random.rand(200, n_actions)

history = model.fit(X_train, y_train, epochs=10, batch_size=16, verbose=1)

# Predict Q-values for a new state
new_state = np.random.rand(1, state_size)
q_values = model.predict(new_state)
print("Predicted Q-values for the new state:", q_values)
```

Output / Result:
The model summary lists two hidden Dense layers of 24 neurons each and an output layer with 2 linear units. Training runs for 10 epochs with the MSE loss decreasing across epochs, and the final prediction step outputs a Q-value for each of the 2 possible actions for a new, unseen state, demonstrating a neural network approximating a value function.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Solution:
Procedure:
1. Import Gymnasium, TensorFlow, and Keras, and create the CartPole-v1 environment.
2. Build a small Q-network using Keras Dense layers that maps a state to Q-values for each action.
3. Implement an epsilon-Greedy policy for action selection and an experience replay buffer to store transitions.
4. Train the network using the Bellman target: target = reward + gamma * max(Q(next_state)), updating the network by minimizing the MSE between predicted and target Q-values.
5. Run the agent for a number of episodes, decaying epsilon over time, and record the reward obtained per episode.
6. Evaluate the trained agent by measuring the average episode reward and the success rate (episodes that reach the maximum step limit).

Program:
```
import random
from collections import deque
import numpy as np
import gymnasium as gym
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
n_actions = env.action_space.n
```

```
model = keras.Sequential([
    layers.Input(shape=(state_size,)),
    layers.Dense(24, activation="relu"),
    layers.Dense(24, activation="relu"),
    layers.Dense(n_actions, activation="linear"),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss="mse")

memory = deque(maxlen=2000)
gamma = 0.95
epsilon, epsilon_min, epsilon_decay = 1.0, 0.01, 0.995
episodes = 50
batch_size = 32
episode_rewards = []

def choose_action(state):
    if np.random.rand() <= epsilon:
        return env.action_space.sample()
    q_values = model.predict(state[np.newaxis], verbose=0)
    return np.argmax(q_values[0])

def replay():
    if len(memory) < batch_size:
        return
    batch = random.sample(memory, batch_size)
    for state, action, reward, next_state, done in batch:
        target = reward
        if not done:
            target += gamma * np.max(model.predict(next_state[np.newaxis], verbose=0)[0])
        target_f = model.predict(state[np.newaxis], verbose=0)
        target_f[0][action] = target
        model.fit(state[np.newaxis], target_f, epochs=1, verbose=0)

for ep in range(episodes):
    state, info = env.reset()
    total_reward = 0
    done = False
    while not done:
        action = choose_action(state)
        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        memory.append((state, action, reward, next_state, done))
        state = next_state
        total_reward += reward
    replay()
    if epsilon > epsilon_min:
        epsilon *= epsilon_decay
    episode_rewards.append(total_reward)
    print(f"Episode {ep+1}/{episodes}: reward={total_reward}, epsilon={epsilon:.3f}")

success_rate = sum(r >= 195 for r in episode_rewards) / episodes * 100
print(f"\nAverage reward: {np.mean(episode_rewards):.2f}")
print(f"Success rate (reward >= 195): {success_rate:.1f}%")
```

Output / Result:

Training prints the reward and the decaying epsilon value for every episode. As training progresses, the episode reward generally trends upward as the agent learns to balance the pole for longer. At the end, the program reports the average reward across all episodes and the success rate (percentage of episodes where the pole was balanced for close to the maximum step count), summarizing the trained agent's learning performance on CartPole.

Code of Conduct:

I ***DUDEKULA MOHAMMAD ILYAS -192325017*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ILYAS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand